ASSESEMENT TOOL-8

ANTONY MAGRIN J(192572103)

1. Online Shopping Platform – Normalize to 3NF

Given Relation

The online shopping platform stores:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Given Table

OrderID	CustID	CustName	ProdID	ProdName	Qty	Price
O101	C01	Asha	P01	Laptop	1	50000
O101	C01	Asha	P02	Mouse	2	800
O102	C02	Rahul	P01	Laptop	1	50000

The same customer and product information is repeated.

Step 1: Identify Functional Dependencies

No.	Functional Dependency	Meaning
1	OrderID → CustID	Order determines customer
2	CustID → CustName	Customer ID determines customer name
3	ProdID → ProdName, Price	Product ID determines product details
4	(OrderID, ProdID) → Qty	Order + Product determines quantity

Step 2: Find Candidate Key

One order can contain many products.

Therefore, OrderID alone cannot uniquely identify a row.

ProdID alone also cannot uniquely identify a row.

Hence:

Candidate Key = (OrderID, ProdID)

Step 3: Convert to 1NF

A relation is in **1NF** if:

- All values are atomic.
- There are no repeating groups.
- Each cell contains a single value.

The given table contains atomic values.

Therefore:

The relation is in 1NF.

Step 4: Convert to 2NF

A relation is in 2NF if:

1. It is already in 1NF.
2. There is no partial dependency.

The candidate key is:

(OrderID, ProdID)

But:

OrderID → CustID

and:

ProdID → ProdName, Price

These attributes depend only on part of the composite key.

Therefore, the relation violates 2NF.

Decompose into 2NF

Order Table

OrderID	CustID	CustName
O101	C01	Asha
O102	C02	Rahul

Product Table

ProdID	ProdName	Price
P01	Laptop	50000
P02	Mouse	800

OrderItem Table

OrderID	ProdID	Qty
O101	P01	1
O101	P02	2
O102	P01	1

Step 5: Convert to 3NF

Now consider the Order table:

OrderID	CustID	CustName
O101	C01	Asha
O102	C02	Rahul

Functional dependencies:

OrderID → CustID

CustID → CustName

Therefore:

OrderID → CustID → CustName

This is a transitive dependency.

Therefore, it violates 3NF.

Remove the Transitive Dependency

Customer Table

CustID	CustName
C01	Asha
C02	Rahul

Order Table

OrderID	CustID
O101	C01
O102	C02

Product Table

ProdID	ProdName	Price
P01	Laptop	50000
P02	Mouse	800

OrderItem Table

OrderID	ProdID	Qty
O101	P01	1
O101	P02	2
O102	P01	1

Final 3NF Design

Table	Primary Key
Customer(CustID, CustName)	CustID
Order(OrderID, CustID)	OrderID
Product(ProdID, ProdName, Price)	ProdID
OrderItem(OrderID, ProdID, Qty)	OrderID + ProdID

Why this is in 3NF

- Customer details are stored only once.
- Product details are stored only once.
- No partial dependency remains.
- No transitive dependency remains.
- Data redundancy is reduced.

Anomalies Removed

Insertion anomaly: A new customer or product can be inserted independently.

Update anomaly: Customer name needs to be changed only once.

Deletion anomaly: Deleting an order does not delete customer information.

Conclusion

The original Order relation is normalized from 1NF → 2NF → 3NF by removing partial and transitive dependencies. The final design contains Customer, Order, Product, and OrderItem tables.

2. Hospital Patient System – Normalize to BCNF

Given Relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Given Table

PID	PName	DocID	DocName	Specialization	WardNo	WardName
P01	Arun	D01	Dr. Ravi	Cardiology	W01	General
P02	Priya	D01	Dr. Ravi	Cardiology	W02	ICU
P03	Kumar	D02	Dr. Meena	Neurology	W01	General

Step 1: Functional Dependencies

No.	Functional Dependency
1	PID → PName, DocID, WardNo
2	DocID → DocName, Specialization
3	WardNo → WardName

Candidate Key

PID

because PID uniquely identifies a patient.

Step 2: Identify Anomalies

Update Anomaly

If Dr. Ravi's specialization changes, it must be updated in every patient record associated with Dr. Ravi.

Insertion Anomaly

A new doctor cannot be stored unless a patient is assigned to that doctor.

Deletion Anomaly

If the last patient assigned to a doctor is deleted, the doctor's information may also be lost.

Step 3: 1NF

All values are atomic.

Therefore:

Patient is in 1NF.

Step 4: 2NF

The candidate key is a single attribute PID.

Therefore, partial dependency cannot occur.

Patient is in 2NF.

Step 5: 3NF

We have:

PID → DocID

DocID → DocName, Specialization

Therefore:

PID → DocID → DocName, Specialization

This is a transitive dependency.

Also:

PID → WardNo

WardNo → WardName

Therefore:

PID → WardNo → WardName

So the original relation is not in 3NF.

Step 6: BCNF

BCNF requires:

For every functional dependency X → Y, X must be a superkey.

But:

DocID → DocName, Specialization

Here DocID is not a superkey of the original relation.

Also:

WardNo → WardName

Here WardNo is not a superkey.

Therefore, the original relation violates BCNF.

Step 7: Decomposition

Patient Table

PID	PName	DocID	WardNo
P01	Arun	D01	W01
P02	Priya	D01	W02
P03	Kumar	D02	W01

Doctor Table

DocID	DocName	Specialization
D01	Dr. Ravi	Cardiology
D02	Dr. Meena	Neurology

Ward Table

WardNo	WardName
W01	General
W02	ICU

Final BCNF Design

Relation	Primary Key
Patient(PID, PName, DocID, WardNo)	PID
Doctor(DocID, DocName, Specialization)	DocID
Ward(WardNo, WardName)	WardNo

BCNF Verification

- In Patient: PID → PName, DocID, WardNo; PID is the key.
- In Doctor: DocID → DocName, Specialization; DocID is the key.
- In Ward: WardNo → WardName; WardNo is the key.

Therefore, every determinant is a superkey.

Conclusion

The hospital relation is decomposed into Patient, Doctor and Ward tables and normalized up to BCNF, eliminating redundancy and insertion, deletion and update anomalies.

3. University Enrollment – 2NF Decomposition

Given Relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Given Table

SID	SName	CourseID	CourseName	Faculty	Grade
S01	Asha	C01	DBMS	Dr. Kumar	A
S01	Asha	C02	OS	Dr. Ravi	B
S02	Rahul	C01	DBMS	Dr. Kumar	A

Step 1: Functional Dependencies

No.	Functional Dependency
1	SID → SName
2	CourseID → CourseName, Faculty
3	(SID, CourseID) → Grade

Step 2: Candidate Key

A student can take multiple courses.

A course can have multiple students.

Therefore:

Candidate Key = (SID, CourseID)

Step 3: 1NF

All values are atomic.

Therefore, the relation is in **1NF**.

Step 4: Check 2NF

Candidate key:

(SID, CourseID)

But:

SID → SName

So SName depends only on SID.

Also:

CourseID → CourseName, Faculty

So CourseName and Faculty depend only on CourseID.

These are partial dependencies.

Therefore, the original relation is not in 2NF.

Step 5: Decompose into 2NF

Student Table

SID	SName
S01	Asha
S02	Rahul

Course Table

CourseID	CourseName	Faculty
C01	DBMS	Dr. Kumar
C02	OS	Dr. Ravi

Enrollment Table

SID	CourseID	Grade
S01	C01	A
S01	C02	B
S02	C01	A

2NF Verification

Student

SID → SName

Course

CourseID → CourseName, Faculty

Enrollment

(SID, CourseID) → Grade

Now there is no partial dependency.

Final 2NF Design

Relation	Primary Key
Student(SID, SName)	SID
Course(CourseID, CourseName, Faculty)	CourseID
Enrollment(SID, CourseID, Grade)	SID + CourseID

Conclusion

The original Enrollment relation is decomposed into Student, Course and Enrollment tables to remove partial dependencies and achieve 2NF.

4. Airline Booking – Normalize to BCNF

Given Relation

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Given Table

FlightNo	PassID	PassName	Seat	DeptCity	ArrCity	Date
F101	P01	Asha	12A	Chennai	Delhi	12-08-26
F101	P02	Rahul	12B	Chennai	Delhi	12-08-26
F102	P01	Asha	10A	Chennai	Mumbai	13-08-26

Functional Dependencies

No.	Functional Dependency
1	PassID → PassName
2	FlightNo → DeptCity, ArrCity
3	(FlightNo, PassID) → Seat, Date

Candidate Key

(FlightNo, PassID)

1NF

All values are atomic.

Therefore, it is in 1NF.

2NF

The key is (FlightNo, PassID).

But:

PassID → PassName

and:

FlightNo → DeptCity, ArrCity

These are partial dependencies.

Therefore, the relation violates 2NF.

Decomposition

Passenger Table

PassID	PassName
P01	Asha
P02	Rahul

Flight Table

FlightNo	DeptCity	ArrCity
F101	Chennai	Delhi
F102	Chennai	Mumbai

Booking Table

FlightNo	PassID	Seat	Date
F101	P01	12A	12-08-26
F101	P02	12B	12-08-26
F102	P01	10A	13-08-26

BCNF Verification

Passenger

PassID → PassName

PassID is the key.

Flight

FlightNo → DeptCity, ArrCity

FlightNo is the key.

Booking

(FlightNo, PassID) → Seat, Date

The composite determinant is the key.

Therefore, all relations satisfy BCNF.

Conclusion

The airline booking relation is decomposed into Passenger, Flight and Booking and normalized up to BCNF.

5. HR System – Normalize to 3NF

Given Relation

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Salary)

Given Table

EmpID	EmpName	DeptID	DeptName	ManagerID	ManagerName	Salary
E01	Asha	D01	CSE	M01	Kumar	50000
E02	Rahul	D01	CSE	M01	Kumar	45000
E03	Priya	D02	ECE	M02	Ravi	55000

Functional Dependencies

No.	Functional Dependency
1	EmpID → EmpName, DeptID, ManagerID, Salary
2	DeptID → DeptName, ManagerID
3	ManagerID → ManagerName

Candidate Key

EmpID

Transitive Dependencies

We have:

EmpID → DeptID

DeptID → DeptName

Therefore:

EmpID → DeptID → DeptName

Similarly:

EmpID → ManagerID

ManagerID → ManagerName

Therefore:

EmpID → ManagerID → ManagerName

These are transitive dependencies.

Decomposition

Employee Table

EmpID	EmpName	DeptID	ManagerID	Salary
E01	Asha	D01	M01	50000
E02	Rahul	D01	M01	45000
E03	Priya	D02	M02	55000

Department Table

DeptID	DeptName	ManagerID
D01	CSE	M01
D02	ECE	M02

Manager Table

ManagerID	ManagerName
M01	Kumar
M02	Ravi

3NF Verification

- Employee: EmpID → EmpName, DeptID, ManagerID, Salary
- Department: DeptID → DeptName, ManagerID
- Manager: ManagerID → ManagerName

The transitive dependencies have been removed from Employee.

Conclusion

The HR schema is normalized to **3NF** by separating Employee, Department and Manager information.

6. Library – Identify MVDs and Decompose to 4NF

Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Given Table

MemberID	MemberName	BookID	Title	Author	Genre	BorrowDate
M01	Asha	B01	DBMS	Korth	Technical	10-08-26
M01	Asha	B02	OS	Silberschatz	Technical	11-08-26
M01	Asha	B03	Novel	Author A	Fiction	12-08-26

Functional Dependencies

MemberID → MemberName

BookID → Title, Author

Multivalued Dependencies

Assume a member can have multiple independent books and genres.

Therefore:

MemberID →→ BookID

and

MemberID →→ Genre

These are **multivalued dependencies (MVDs)**.

4NF Rule

A relation is in 4NF if:

For every non-trivial MVD X →→ Y, X must be a superkey.

Here MemberID is not a superkey for the whole relation.

Therefore, the relation violates **4NF**.

Decomposition

Member Table

MemberID	MemberName
M01	Asha

Book Table

BookID	Title	Author
B01	DBMS	Korth
B02	OS	Silberschatz
B03	Novel	Author A

MemberBook Table

MemberID	BookID	BorrowDate
M01	B01	10-08-26
M01	B02	11-08-26
M01	B03	12-08-26

MemberGenre Table

MemberID	Genre
M01	Technical
M01	Fiction

Conclusion

The independent multivalued dependencies are separated into different relations. Therefore, the database is decomposed into relations satisfying 4NF, reducing redundancy caused by independent multivalued facts.

7. Telecom Billing – Candidate Keys, Primary Key and FDs

Given Relation

Billing(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDuration, Amount)

Given Table

BillID	CustomerID	CustomerName	PhoneNo	PlanID	PlanName	CallID	CallDuration	Amount
B01	C01	Asha	98765	PL01	Basic	CL01	10	100
B01	C01	Asha	98765	PL01	Basic	CL02	20	200
B02	C02	Rahul	91234	PL02	Premium	CL03	15	180

Functional Dependencies

No.	FD
1	BillID → CustomerID, PhoneNo, PlanID
2	CustomerID → CustomerName
3	PlanID → PlanName
4	CallID → CallDuration
5	(BillID, CallID) → Amount

Candidate Keys

Depending on the business rule:

- BillID identifies a bill.
- (BillID, CallID) identifies a billing detail/call record.

For the complete bill-detail relation, a suitable candidate key is:

(BillID, CallID)

Primary Key

If the system treats each bill as uniquely identified by BillID, then:

Primary Key = BillID

For individual call details:

Composite Key = (BillID, CallID)

Partial Dependencies

If (BillID, CallID) is the key:

BillID → CustomerID, PhoneNo, PlanID

These depend only on BillID, which is part of the composite key.

Therefore, partial dependencies exist.

Transitive Dependencies

BillID → CustomerID

CustomerID → CustomerName

Therefore:

BillID → CustomerID → CustomerName

Similarly:

BillID → PlanID

PlanID → PlanName

Therefore:

BillID → PlanID → PlanName

Possible Normalized Design

Customer

CustomerID	CustomerName
C01	Asha
C02	Rahul

Plan

PlanID	PlanName
PL01	Basic
PL02	Premium

Bill

BillID	CustomerID	PhoneNo	PlanID
B01	C01	98765	PL01
B02	C02	91234	PL02

Call

CallID	CallDuration
CL01	10
CL02	20
CL03	15

BillCall

BillID	CallID	Amount
B01	CL01	100
B01	CL02	200
B02	CL03	180

Conclusion

Before normalization, the important tasks are to identify candidate keys, primary keys, functional dependencies, partial dependencies and transitive dependencies. These dependencies guide further normalization.

8. Retail Inventory – Complete Normalization

Given Relation

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Given Table

PID	PName	SupplierID	SupplierName	Category	Price	Warehouse
P01	Laptop	S01	ABC Ltd	Electronics	50000	W01
P02	Mouse	S01	ABC Ltd	Electronics	800	W01
P03	Keyboard	S02	XYZ Ltd	Electronics	1500	W02

Functional Dependencies

No.	FD
1	PID → PName, SupplierID, Category, Price, Warehouse
2	SupplierID → SupplierName

Candidate Key

PID

1NF

All values are atomic.

Therefore:

Product is in 1NF.

2NF

The candidate key consists of only one attribute:

PID

Therefore, partial dependency is impossible.

So:

Product is in 2NF.

3NF

We have:

PID → SupplierID

and:

SupplierID → SupplierName

Therefore:

PID → SupplierID → SupplierName

This is a transitive dependency.

Therefore, the original relation violates **3NF**.

Decomposition

Product Table

PID	PName	SupplierID	Category	Price	Warehouse
P01	Laptop	S01	Electronics	50000	W01
P02	Mouse	S01	Electronics	800	W01
P03	Keyboard	S02	Electronics	1500	W02

Supplier Table

SupplierID	SupplierName
S01	ABC Ltd
S02	XYZ Ltd

BCNF Check

Product

PID → PName, SupplierID, Category, Price, Warehouse

PID is a superkey.

Supplier

SupplierID → SupplierName

SupplierID is a superkey.

Therefore, the resulting relations satisfy BCNF under the given dependencies.

Conclusion

The Product relation is normalized through:

1NF → 2NF → 3NF, and the resulting relations also satisfy BCNF under the stated dependencies.

9. School ERP – Normalize to BCNF

Given Relation

School(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Given Table

StudentID	StudentName	ClassID	ClassName	TeacherID	TeacherName	SubjectID	SubjectName	Grade
S01	Asha	C01	II CSE	T01	Kumar	SUB01	DBMS	A
S01	Asha	C01	II CSE	T01	Kumar	SUB02	OS	B
S02	Rahul	C01	II CSE	T01	Kumar	SUB01	DBMS	A

Functional Dependencies

No.	Functional Dependency
1	StudentID → StudentName, ClassID
2	ClassID → ClassName, TeacherID
3	TeacherID → TeacherName
4	SubjectID → SubjectName
5	(StudentID, SubjectID) → Grade

Candidate Key

A student can study multiple subjects.

Therefore:

Candidate Key = (StudentID, SubjectID)

1NF

All values are atomic.

Therefore, the relation is in 1NF.

2NF

StudentID → StudentName, ClassID

and:

SubjectID → SubjectName

These depend on only part of the composite key.

Therefore, the relation violates 2NF.

3NF

We have:

StudentID → ClassID

ClassID → ClassName, TeacherID

Therefore:

StudentID → ClassID → ClassName, TeacherID

Also:

TeacherID → TeacherName

Therefore:

StudentID → ClassID → TeacherID → TeacherName

These are transitive dependencies.

BCNF Decomposition

Student Table

StudentID	StudentName	ClassID
S01	Asha	C01
S02	Rahul	C01

Class Table

ClassID	ClassName	TeacherID
C01	II CSE	T01

Teacher Table

TeacherID	TeacherName
T01	Kumar

Subject Table

SubjectID	SubjectName
SUB01	DBMS
SUB02	OS

Enrollment Table

StudentID	SubjectID	Grade
S01	SUB01	A
S01	SUB02	B
S02	SUB01	A

BCNF Verification

Relation	Functional Dependency	Determinant
Student	StudentID → StudentName, ClassID	StudentID is key
Class	ClassID → ClassName, TeacherID	ClassID is key
Teacher	TeacherID → TeacherName	TeacherID is key
Subject	SubjectID → SubjectName	SubjectID is key
Enrollment	(StudentID, SubjectID) → Grade	Composite key

Every determinant is a superkey.

Therefore, the design satisfies BCNF.

Conclusion

The poorly normalized School ERP is decomposed into Student, Class, Teacher, Subject and Enrollment tables and normalized up to BCNF.

10. Movie Streaming Platform – 5NF

Given Relation

Consider:

Streaming(UserID, MovieID, ActorID)

Example Table

UserID	MovieID	ActorID
U01	M01	A01
U01	M01	A02
U02	M01	A01
U02	M02	A03

Assume the system maintains three independent relationships:

1. User watches Movie.
2. Actor appears in Movie.
3. User has a relationship with Actor.

Step 1: Identify Join Dependency

The original relation:

Streaming(UserID, MovieID, ActorID)

can be represented using three projections:

UserMovie

UserID	MovieID
U01	M01
U02	M01
U02	M02

MovieActor

MovieID	ActorID
M01	A01
M01	A02
M02	A03

UserActor

UserID	ActorID
U01	A01
U01	A02
U02	A01
U02	A03

The join dependency is:

Streaming = UserMovie ⋈ MovieActor ⋈ UserActor

Step 2: Why 5NF?

A relation is in 5NF when every non-trivial join dependency is implied by candidate keys.

The original relation combines three independent relationships.

This may create unnecessary combinations and redundancy.

Therefore, it should be decomposed.

Step 3: 5NF Decomposition

UserMovie Table

UserID	MovieID
U01	M01
U02	M01
U02	M02

MovieActor Table

MovieID	ActorID
M01	A01
M01	A02
M02	A03

UserActor Table

UserID	ActorID
U01	A01
U01	A02
U02	A01
U02	A03

Step 4: Lossless Join

The original relation can be reconstructed as:

UserMovie ⋈ MovieActor ⋈ UserActor

Therefore, when the assumed join dependency is valid, the decomposition is lossless.

The decomposition separates:

- User–Movie relationship
- Movie–Actor relationship
- User–Actor relationship

Step 5: Advantages

1. Reduces redundancy.
2. Avoids unnecessary combinations.
3. Simplifies insertion.
4. Simplifies deletion.
5. Maintains independent relationships.
6. Provides a lossless decomposition under the stated join dependency.

Conclusion

The Movie Streaming relation contains a join dependency and is decomposed into UserMovie, MovieActor and UserActor. The original relation can be reconstructed using natural joins, making the decomposition lossless and satisfying 5NF under the stated business rules.